

Industrial Internship Report on "Multi-Client Service Platform (Task Management System)"

Prepared by

Mahumood Sameema S

Executive Summary

This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT).

This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks' time.

My project was the development of a Multi-Client Service Platform (Task Management System) that allows multiple users to manage and schedule tasks efficiently using a full stack architecture.

This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1	Preface	3
2	Introduction	4
2.1	About UniConverge Technologies Pvt Ltd	4
2.2	About upskill Campus	8
2.3	Objective	10
2.4	Reference	10
2.5	Glossary	10
3	Problem Statement	11
4	Existing and Proposed solution	12
5	Proposed Design/ Model	14
5.1	High Level Diagram (if applicable)	15
5.2	Low Level Diagram (if applicable)	15
5.3	Interfaces (if applicable)	15
6	Performance Test	17
6.1	Test Plan/ Test Cases	17
6.2	Test Procedure	18
6.3	Performance Outcome	18
7	My learnings	19
8	Future work scope	20

1 Preface

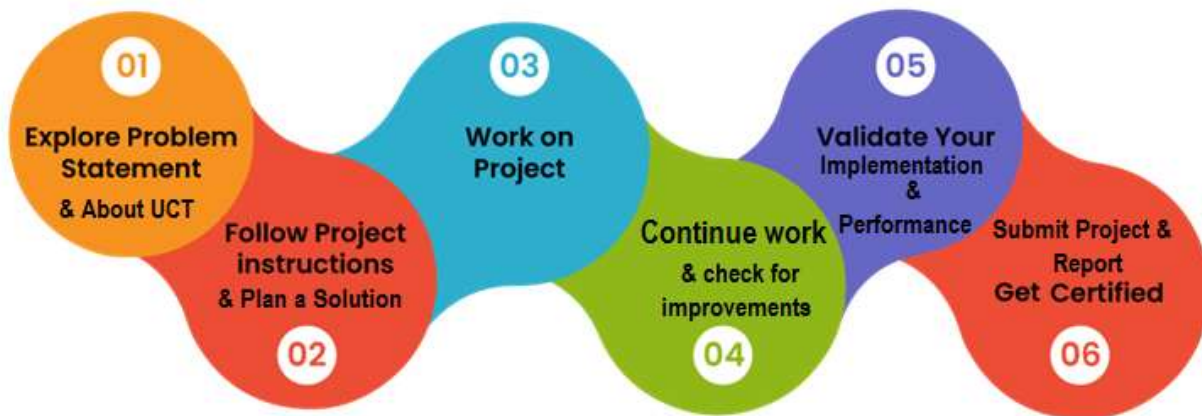
Summary of the whole 6 weeks' work.

About need of relevant Internship in career development.

Brief about Your project/problem statement.

Opportunity given by USC/UCT.

How Program was planned



Your Learnings and overall experience.

Thank to all (with names), who have helped you directly or indirectly.

Your message to your juniors and peers.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies** e.g. **Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.



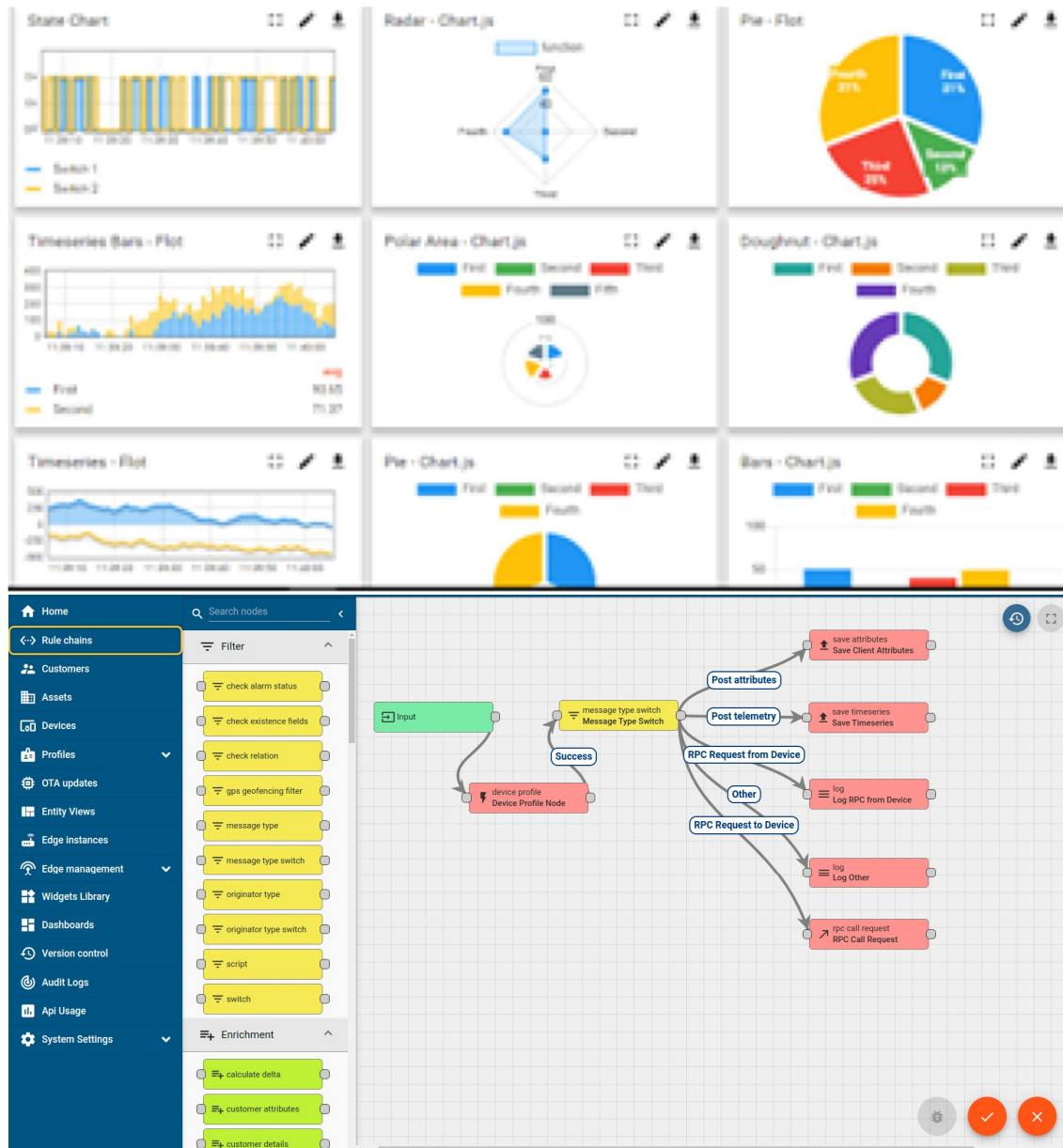
i. UCT IoT Platform ()

UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable “insight” for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine



FACTORY WATCH

ii. Smart Factory Platform ()

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleash the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they want to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.



Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i



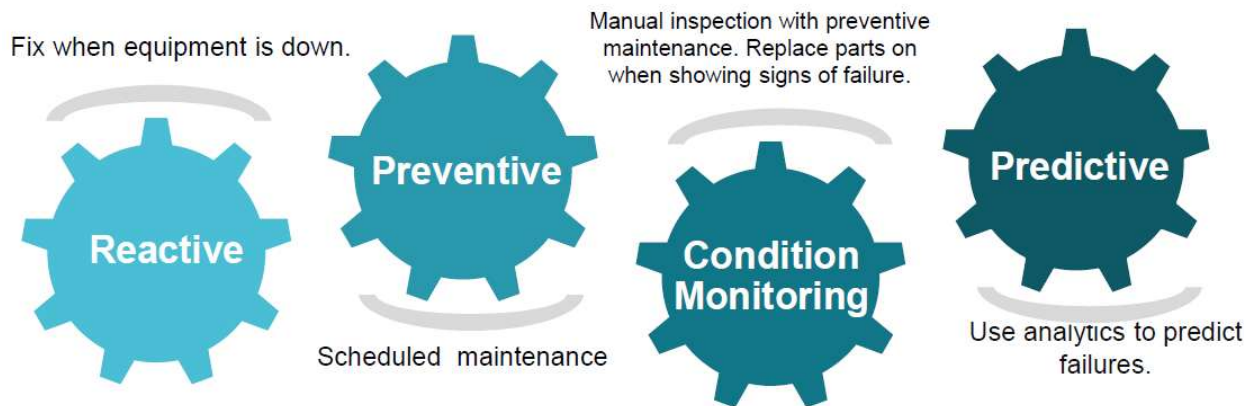


iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN technology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

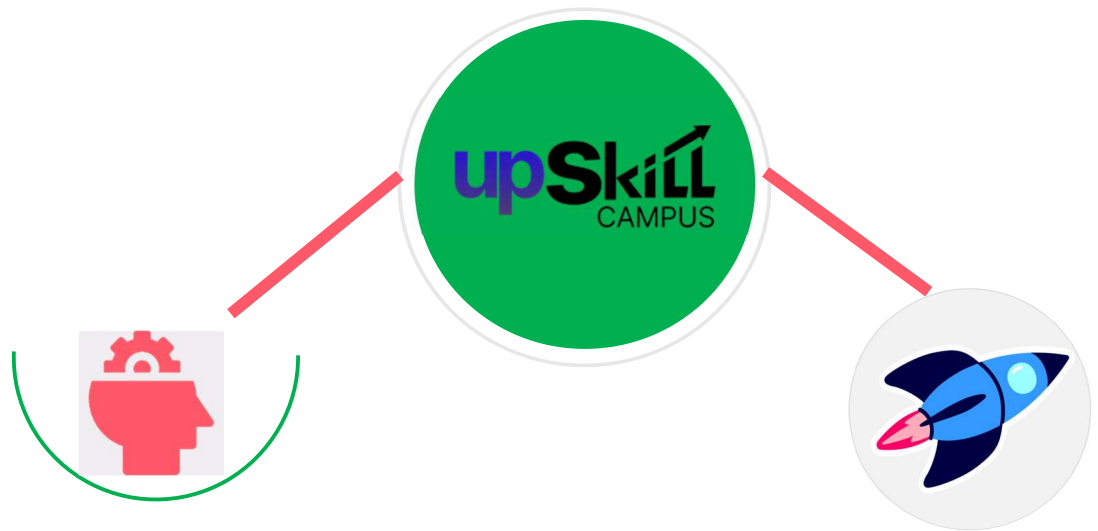
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

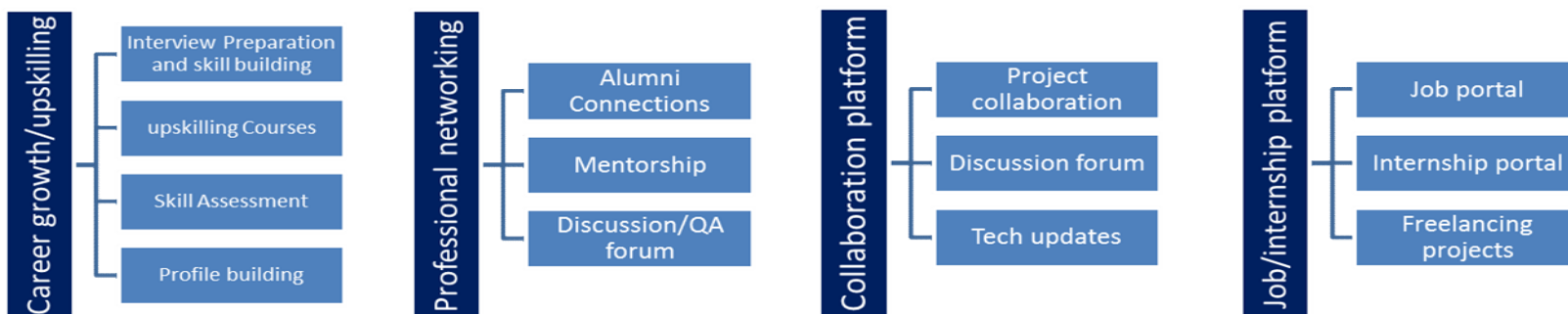
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

<https://www.upskillcampus.com/>



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- get practical experience of working in the industry.
- to solve real world problems.
- to have improved job prospects.
- to have Improved understanding of our field and its applications.
- to have Personal growth like better communication and problem solving.

2.5 Reference

- [1] MongoDB official documentation
- [2] Online tutorials and learning resources from Upskill Campus
- [3] Java (J2SE) concepts and study materials provided during the internship

2.6 Glossary

Terms	Acronym
Multi-Client System	A system that allows multiple users to access and use the application independently.
Frontend	The user interface of the application that users interact with.
Backend	The server-side logic that processes user requests and manages data.
Database	A system used to store and manage data (MongoDB in this project).
Authentication	The process of verifying user identity during login.

3 Problem Statement

In the assigned problem statement

In the current fast-paced world, efficiently managing tasks is a common problem faced when working with multiple users. Many individuals face difficulties in organizing their tasks efficiently.

Most systems do not allow the users to efficiently schedule their tasks or do not provide a good view of the tasks through dashboards.

Thus, there is a need to create a Multi-Client Service Platform through which users can efficiently manage their tasks individually, can efficiently schedule their tasks through intelligent scheduling, and can easily view their tasks through a dashboard or a calendar view.

4 Existing and Proposed solution

Existing Solutions

There are a number of task management and productivity tools available, ranging from simple task list management tools to calendar-based task management tools. While these tools enable users to effectively manage tasks, there are a number of limitations associated with the tools.

Most of the existing solutions:

Do not effectively handle multiple users with independent task management requirements

Do not have intelligent task scheduling mechanisms

Do not have good visualization of tasks for different dates

Do not have a unified tool for daily tasks, planning, and calendar management

Shortcomings of the existing solutions

Lack of support for multi-client (multi-user) systems

Manual allocation of tasks without the help of automation

No integration between task planning and calendar view

Inability to efficiently monitor tasks over a long period of time

Proposed Solution

In the proposed system, the Multi-Client Service Platform is implemented as a Task Management System. The system supports multiple users to register themselves, log in, and manage their tasks individually.

The system includes the following features:

A Today Dashboard to monitor tasks on a daily basis

A Task Planning feature to manage tasks

A Calendar View to monitor tasks over a range of dates

A simple planner algorithm to assign tasks to the next day

4.1 Code submission (Github link)

<https://github.com/Mahumood-Sameema/upskillcampus>

4.2 Report submission (Github link) :

<https://github.com/Mahumood-Sameema/upskillcampus>

5 Proposed Design/ Model

The design flow of the proposed system is structured in accordance with a client-server design paradigm. The design is intended to support multiple users and effectively manage tasks in various stages.

1. Input Stage (User Interaction)

- Users register and log in to the system.
- Users input their tasks, which include the name and schedule.
- User request is sent to the backend from the frontend.

2. Processing Stage

- User request is processed by the backend.
- Tasks are saved in the database with user-specific details.
- Task assignment is done by the planner either manually or automatically to the next available day.
- Tasks are organized in a structured manner according to the date and users.

3. Intermediate Stage

- Tasks are grouped into categories, which include:
- Tasks to be done today.
- Tasks to be done in the future (using a calendar).
- Task status is updated dynamically.
- Tasks are updated and deleted.

4. Output Stage (User Interface)

- Processed data is displayed to users on the frontend.
- Users are allowed to:
- View today's dashboard.
- View the task planning section.
- View the calendar.

5. System Flow Summary

- User logs in to the system.
- User creates/manages their tasks.
- Backend processes and saves user request.
- Tasks are scheduled and organized.
- Tasks are displayed to users in a dashboard and calendar.

5.1 High Level Diagram (if applicable)

User → Frontend (React UI) → Backend (Node.js/Express) → Database (MongoDB)

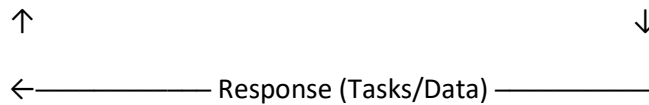


Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

5.2 Low Level Diagram (if applicable)

5.3 Interfaces (if applicable)

The system has various interfaces that facilitate communication between different parts of the application.

1. User Interface (Frontend)

- Developed using: React.js
- It contains various pages such as:
 1. Today Dashboard
 2. Task Planning
 3. Calendar View
 4. Users can interact with the system by creating, updating, and viewing tasks.

2. Backend Interface

- Developed using: Node.js + Express.js
- This interface handles all the requests sent by the users to the system.
- It processes the data and manages the application.
- It ensures proper communication between the frontend and the database.

3. Database Interface

- MongoDB is used to store data.
- It contains information such as:
 - User information
 - Task information
 - Schedules

4. Data Flow

The data flow in the system is as follows:

- User interacts with the system through the frontend.
- The request is sent to the backend by the frontend using API calls.
- The backend processes the request.
- The backend interacts with the database.
- The response is sent back to the frontend.
- The data is shown to the users.

5. Communication Protocol

- HTTP/HTTPS protocol is used for communication between the frontend and backend.
- RESTful API is used to exchange data.

6 Performance Test

The performance of the "Multi Client Service Platform" can be measured based on certain constraints like "response time," "data handling capacity," and "scalability."

1. Identified Constraints

The system is based on certain identified constraints as follows:

- Response Time: The time taken to load pages and fetch data for tasks
- Data Handling Capacity: The ability to handle multiple users and tasks
- Scalability: The ability to handle an increasing number of users
- Network Dependency: The system's dependency on internet connectivity
- Memory Usage: The ability to efficiently store and retrieve task data

2. Handling of Constraints in Design

The system design handles these constraints in the following manner:

- Efficient API Communication: Faster response due to efficient API communication
- MongoDB Database Usage: Efficient data storage and retrieval using MongoDB
- Modular Design: Ability to scale the system for handling multiple users
- Structured Format for Task Management: Efficient task management in a structured format to avoid data redundancy
- Frontend Design: Lightweight design to enhance loading speed

3. Test Results

The system was tested under normal conditions as follows:

- The system responds quickly to user actions
- Multiple users can independently manage tasks
- Efficient data storage and retrieval
- No delays were experienced during normal operations

6.1 Test Plan/ Test Cases

The system is tested to check for proper functionality and performance. The following test cases were considered for testing the system:

- User Registration Test: Check whether users can register successfully
- User Login Test: Check whether users can log in to the system

- Task Creation Test: Check whether tasks can be created successfully
- Task Update/Delete Test: Check whether tasks can be updated or deleted
- Task Scheduling Test: Check whether tasks are scheduled properly (manual and automatic)
- Dashboard Display Test: Check whether tasks for today are properly displayed
- Calendar View Test: Check whether tasks are properly displayed for the date

6.2 Test Procedure

The process of testing the application is as follows:

- The application was run in a local environment
- Users were registered and logged into the application
- Tasks were created, updated, and deleted to test CRUD operations
- The planner feature of the application was tested by auto-assigning tasks
- The dashboard and calendar features of the application were used to check if tasks are displayed correctly
- The application was tested for various user scenarios to check if data is handled independently

6.3 Performance Outcome

The process of testing the application is as follows:

- The application was run on a local environment
- Users were registered and logged into the application
- Tasks were created, updated, and deleted to test CRUD operations
- The planner feature of the application was tested by auto-assigning tasks to users
- The dashboard and calendar features of the application were used to test if tasks are displayed properly
- The application was tested for various user scenarios to check if data is handled independently

7 My learnings

During the course of this project, I have learned many important things related to full stack web development and system design. I have learned how to create a full-fledged application with the help of frontend, backend, and database programming.

I have enhanced my knowledge in the following areas:

- Client-server architecture
- API programming and integration
- Database programming with MongoDB
- User authentication and task management programming

I have also learned how to design a project, manage data, and create real-world functionality like task scheduling and user handling.

This project has helped me to enhance my problem-solving abilities and knowledge, which is important for my future career in software programming and full stack engineering.

8 Future work scope

Despite the fact that the system works efficiently, there are several enhancements that can be made to the system in the future. These enhancements are as follows:

- Improve the task scheduling algorithm to be more intelligent
- Add notifications/reminder features to tasks
- Add role-based access control for different categories of users
- Improve the UI/UX of the application to be user-friendly
- Deploy the application on cloud platforms to make it more practical
- Add mobile application support to the application

These enhancements will help to make the system more efficient and practical.